

Comparative Study of Value-Based and Policy-Gradient Reinforcement Learning on ARC-AGI-3

School of Computer Science & Applied Mathematics
University of the Witwatersrand

COMS4061A/COMS7071A: Reinforcement Learning

Tebogo Seopa (2563912)
Mandisa Mashinini (2589970)
Onkabetse Monamodi (2854004)

September 17, 2026

Abstract

ARC-AGI-3 is an interactive reasoning benchmark in which agents must discover an unfamiliar game’s rules and goals purely through interaction, with no explicit instructions and no native scalar reward beyond level completion. This project implements and systematically improves two learning agents for this benchmark: a value-based baseline, DQN, and a policy-gradient baseline, PPO. It investigates which kinds of improvement (exploration, memory, action-space abstraction, reward shaping, and curriculum) transfer across both algorithm families and generalise to games the agents have not seen before. Both agents are evaluated under a matched, reproducible protocol that uses the competition’s own scoring interface, benchmarked against a random-action reference policy, with at least three evaluated versions of each (baseline, improvement 1, improvement 2). The project is submitted to both ARC Prize 2026 tracks, ARC-AGI-3 and Paper Track, in addition to the course report.

Contents

Preface	
Abstract	i
Table of Contents	ii
1 Introduction	1
1.1 Background	1
1.2 Research Problem	1
1.3 Research Question	2
1.4 Aim	2
1.5 Objectives	2
1.6 Proposed Contribution	2
1.7 Overview of the Proposal	3
2 Related Work	4
2.1 Value-Based Reinforcement Learning	4
2.2 Policy-Gradient Reinforcement Learning	4
2.3 Exploration Under Sparse Reward	4
2.4 Memory and Partial Observability	4
2.5 Curriculum Learning	5
2.6 Prior ARC-AGI-3 Agents	5
2.7 Research Gap	5
3 Research Methodology	6
3.1 Environment	6
3.2 Agents	6
3.3 Improvement Directions	6
3.4 Evaluation Framework	7
3.5 Tools and Implementation	7
4 Schedule of Work	8
5 Conclusion	10
References	10

Chapter 1

Introduction

1.1 Background

Most benchmarks used to teach reinforcement learning (grid worlds, Atari, classic control) give an agent a fixed, well-understood observation and action space, and a dense or at least consistent reward signal. ARC-AGI-3 removes both comforts. Each game is a previously unseen 64×64 , 16-colour grid world, the actions are a mix of five simple discrete moves and one continuous (x, y) click over the grid, and the only external signal is level completion. The platform itself gives no shaped per-step reward. This is deliberate. The benchmark is designed to test exploration, world-modelling, goal inference, and planning under genuine novelty [1], and this is exactly the kind of problem where naive tabular or shallow function-approximation RL struggles, and where exploration strategy, memory, and representation matter more than the choice of algorithm.

Public solutions so far lean heavily on prompting large pretrained vision-language models as the policy. The GPT-OSS-120B-template agents that placed at Milestone 1, for example, render frames as images and ask a VLM to plan and act [2]. Others use hand-crafted click heuristics layered over a small learned change-predictor, such as StochasticGoose, which trains a CNN to bias exploration toward actions likely to change the frame [9]. Neither is a systematically ablated, from-scratch RL investigation of the kind this course requires.

1.2 Research Problem

Classic deep RL algorithms, value-based and policy-gradient, have not been systematically compared and improved on ARC-AGI-3 under matched budgets. It is not established which categories of improvement meaningfully help each algorithm family, or whether gains from one transfer to the other: exploration bonuses for sparse, level-gated reward; recurrent memory for partial observability in an unfamiliar game; curriculum ordering across games of varying difficulty; or reward shaping beyond raw level completion.

1.3 Research Question

This project is organised around the following research question:

Do targeted improvements, specifically exploration shaping, short-horizon memory (frame-stacking or recurrence), and an abstracted click action space for ACTION6, measurably improve a DQN and a PPO agent’s interaction efficiency and generalisation to held-out ARC-AGI-3 games, compared with their own baselines and a random-action reference policy? And do the improvements that help one algorithm also help the other?

1.4 Aim

To implement, evaluate, and iteratively improve one value-based (DQN) and one policy-gradient (PPO) agent for ARC-AGI-3, under a single reproducible evaluation protocol, and to determine which improvement categories generalise across algorithms and across games not seen during development.

1.5 Objectives

1. **Environment integration.** Wrap the official `arc_agi/arcengine` interface (reset/step, `GameAction`, `FrameData`) in a Gymnasium environment so standard RL libraries can train on it; validate against a random-action sanity-check policy.
2. **Baseline implementation.** Implement a DQN baseline (course algorithm) and a PPO baseline (non-course algorithm), each with a clear account of what is learned, when, and how it changes the policy.
3. **Reproducible evaluation protocol.** Fix action budgets, seeds, train/held-out game splits, and the scoring procedure (official levels-completed / efficiency score) before any improvement work begins.
4. **Iterative improvement.** For each algorithm, implement and evaluate at least two substantive, motivated changes (baseline $\rightarrow$ improvement 1 $\rightarrow$ improvement 2), each justified by an observed limitation and a stated hypothesis.
5. **Comparative analysis.** Ablate each improvement’s individual contribution, measure variability across ≥ 3 seeds, and compare the two final agents on interaction efficiency, compute cost, and generalisation to games withheld from development.
6. **Reporting and submission.** Submit working code to the ARC-AGI-3 Kaggle track, a condensed write-up to the Paper Track, and the full RLC-format report to Moodle.

1.6 Proposed Contribution

This project does not propose a new algorithm. Its contribution is a matched, ablated comparison of standard value-based and policy-gradient RL under a novel-environment,

sparse-reward, mixed discrete/continuous action setting. Published ARC-AGI-3 solutions are dominated by VLM-as-policy or heuristic approaches rather than trained RL agents evaluated with proper baselines and ablations, so this comparison fills a real gap.

1.7 Overview of the Proposal

The rest of this proposal is structured as follows. Chapter 2 reviews the background literature and motivates the research problem. Chapter 3 describes the research methodology in detail. Chapter 4 presents the schedule of work. Chapter 5 concludes the proposal.

Chapter 2

Related Work

2.1 Value-Based Reinforcement Learning

DQN [7] established that a convolutional Q-network with experience replay and a target network can learn control policies directly from pixel-like observations, which maps naturally onto ARC-AGI-3’s 64×64 coloured-grid frames.

2.2 Policy-Gradient Reinforcement Learning

PPO [8] is the non-course algorithm for this project; its clipped surrogate objective gives more stable updates than vanilla policy gradients, which is likely relevant given ARC-AGI-3’s non-stationary, per-game reward structure.

2.3 Exploration Under Sparse Reward

Bellemare et al. [4] show that pseudo-count-based intrinsic bonuses substantially improve exploration in sparse-reward settings. This applies directly here, since the only native signal is level completion.

2.4 Memory and Partial Observability

Hausknecht and Stone [6] show that replacing DQN’s stacked-frame input with a recurrent layer (DRQN) improves performance when the environment is only partially observable within a single frame, relevant to ARC-AGI-3 games whose rules must be inferred over several interactions rather than read off one frame.

2.5 Curriculum Learning

Bengio et al. [5] show that ordering training examples from easy to hard improves both convergence speed and final performance, motivating a possible curriculum across ARC-AGI-3 games of varying apparent complexity.

2.6 Prior ARC-AGI-3 Agents

Public Milestone 1 leaderboard entries are predominantly vision-language-model agents that render frames as images and prompt an LLM (e.g. Gemma- or GPT-OSS-120B-based agents) for a short action plan, with hand-built heuristics (dead-signature detection, legal-action constraints) layered on top [2]. StochasticGoose [9] instead trains a small CNN purely to predict which actions will change the frame, biasing exploration toward informative actions without a full RL loop. Neither line of work reports the kind of matched-budget, multi-seed, ablated comparison this course requires.

2.7 Research Gap

Published ARC-AGI-3 approaches split into prompted-VLM agents and hand-tuned exploration heuristics; systematic, ablated deep-RL baselines with documented improvement trajectories are largely absent from public write-ups. This project fills that specific gap for two standard algorithm families rather than proposing a new one.

Chapter 3

Research Methodology

3.1 Environment

ARC-AGI-3 games run through the official `arc_agi` (`Arcade/EnvironmentWrapper`) and `arcengine` (`GameAction/FrameData`) packages, confirmed by direct inspection of the ARC-AGI-3-Agents source [3]: `GameAction` covers RESET plus six discrete actions (`ACTION1–5`, `ACTION7`) and one continuous click action (`ACTION6`, $x, y \in [0, 63]$); `FrameData` returns the current grid, `GameState` (`NOT_FINISHED`/`WIN`/`GAME_OVER`), and `levels_completed`. This is not a Gymnasium environment out of the box, so a custom `gym.Env` adapter (`ArcAgi3Env`, already implemented) wraps `reset()`/`step()` into the standard tuple interface, discretises `ACTION6` onto a coarser click grid to keep the action space tractable, and shapes reward from level-completion deltas. This is a training-time choice, and it will be reported separately from the official evaluation score, as the assignment’s evaluation-integrity requirement demands.

3.2 Agents

- **DQN baseline** (course algorithm): CNN encoder over the grid observation, standard replay buffer and target network [7].
- **PPO baseline** (non-course algorithm): shared CNN encoder with separate policy/value heads, clipped surrogate objective [8].
- **Reference policy**: uniform random action selection, required as a sanity check and floor for both agents.

Both baselines will be implemented with Stable-Baselines3 on top of the Gymnasium adapter, trained locally for fast iteration and on Kaggle’s T4 accelerator for full runs (no local GPU is available).

3.3 Improvement Directions

Two motivated, evaluated improvements per algorithm, drawn from Chapter 2:

1. **Exploration shaping:** a pseudo-count or novelty-based intrinsic bonus [4] to address sparse, level-gated reward.
2. **Short-horizon memory:** frame-stacking or a recurrent layer [6] to address rule and goal inference that unfolds over several actions rather than a single frame.
3. **Curriculum across games** [5] or **action-space refinement** for ACTION6 as a third candidate direction if time and results warrant a third comparison point.

Each change follows: observed limitation in the prior stage $\rightarrow$ hypothesis $\rightarrow$ intervention $\rightarrow$ measured effect, whether or not it improves the score.

3.4 Evaluation Framework

Protocol. Fixed before improvements begin: action budget per episode, evaluation game set held out from development, ≥ 3 seeds per configuration.

Metrics. Official ARC-AGI-3 score (levels completed / action-efficiency), local vs. Kaggle score reported separately, learning curves against interaction budget, wall-clock and compute cost.

Ablations. Each improvement evaluated in isolation and in combination, on the same held-out games, to attribute effect correctly.

Generalisation. Agents evaluated on games never used during training/tuning.

Failure analysis. Representative failed trajectories reported alongside successes, not only best-case runs.

3.5 Tools and Implementation

Python 3.12 (required by the `arc-agi` package), Gymnasium, Stable-Baselines3, the official ARC-AGI-3 Kaggle Starter kit for local iteration (`make play-local`) and Kaggle submission packaging, Kaggle T4 notebooks for training runs, Git for version control. Weights and Biases will be used for experiment tracking if it does not conflict with the Kaggle no-internet constraint during accelerated runs.

Chapter 4

Schedule of Work

Project window: 17 September – 27 October 2026 (Moodle deadline 23:59 SAST). One buffer day is built into each of Phases 3–5 for unexpected training or API issues. Table [4.1](#) on the facing page summarises the planned timeline.

Table 4.1: Project schedule of work

Phase	Period	Activities
1. Environment & protocol	17–21 Sept	Gymnasium adapter (done); verify frame shape/value assumptions against a live game; implement random-policy sanity check; fix the evaluation protocol (seeds, budgets, held-out game split)
2. Baselines	22 Sept – 5 Oct	DQN and PPO baselines via SB3; local iteration then Kaggle T4 runs; multi-seed baseline results and learning curves
3. Improvement round 1	6–12 Oct	First motivated improvement per algorithm; evaluate against baseline; write up limitation $\rightarrow$ hypothesis $\rightarrow$ result for each
4. Improvement round 2 + ablations	13–19 Oct	Second improvement per algorithm; ablation study isolating each component; held-out-game generalisation runs
5. Final comparison & Kaggle submission	20–23 Oct	Cross-algorithm comparison and efficiency/compute analysis; nominate class-leaderboard entry; finalise Kaggle notebook
6. Report and write-up	24–27 Oct	RLC-template report (≤ 8 pages), Paper Track write-up (<1500 words), ZIP assembly, submission

Chapter 5

Conclusion

This project treats ARC-AGI-3 as a genuine RL research problem rather than a competition leaderboard exercise: a matched-budget, ablated comparison of DQN and PPO, each improved through a documented sequence of motivated changes, evaluated for both raw performance and generalisation to unseen games. The expected outcome is not a state-of-the-art score but a clear, evidence-backed account of which RL improvement categories transfer across algorithm families on a benchmark specifically designed to resist naive approaches.

References

- [1] ARC Prize Foundation. ARC Prize 2026 Documentation. 2026. <https://docs.arcprize.org/arc-prize-2026>
- [2] ARC Prize Foundation. ARC Prize 2026: ARC-AGI-3 Milestone Prize #1. 2026. <https://arcprize.org/blog/arc-prize-2026-milestone-1>
- [3] arcprize/ARC-AGI-3-Agents. Source repository (inspected directly for `Agent`, `EnvironmentWrapper`, `GameAction`, `FrameData` semantics). <https://github.com/arcprize/ARC-AGI-3-Agents>
- [4] M. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, and R. Munos. Unifying Count-Based Exploration and Intrinsic Motivation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [5] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum Learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, 2009.
- [6] M. Hausknecht and P. Stone. Deep Recurrent Q-Learning for Partially Observable MDPs. *AAAI Fall Symposium Series*, 2015. <https://arxiv.org/abs/1507.06527>
- [7] V. Mnih, K. Kavukcuoglu, D. Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*, 2017. <https://arxiv.org/abs/1707.06347>
- [9] D. Smit. ARC3-solution: StochasticGoose. 2026. <https://github.com/DriesSmit/ARC3-solution>